AE 4610

Georgia Institute of Technology

Final Project: Rover Cruise Control

Spring 2025

Group A07 - A

Name	Contributions
Erik Goeke	Worked on rover assembly, and worked on Arduino code for the system. Did wiring for the Arduino system with the motor driver and GPS, and did testing on the system when put together. Tuned the experimental values for gain, based on the ideal scenario. Wrote a large amount of the hardware section of the report, and outlined the design of the rover and the implementation of how the experiment was done.
Elijah Isakson	Helped with initial rover assembly, system ID and measurements, mathematical modeling of system dynamics, aided with controller design (tuning gain values), wrote the Summary, Introduction, and System Dynamics Modeling sections of the report
Maksym Kushniryk	Simulation
William Cooksey	Assisted in initial assembly of 'rover' via soldering of wires to motors, collected data relevant to creation of plant for Simulink and simulation of rover movement and system response, via research on rover controller requirements established desired criteria for controller, and detailed process by which desired K_p, K_i, and K_d values were obtained in 'Controller Design' section of report
Brendan Sullivan	Developed equations of motion; hardware pictures, components, testing, calibration, and results; wrote the hardware implementation and controller evaluation and conclusion sections of the report

I. Summary

The goal of this project is to build a small-scale rover model that utilizes a closed-loop feedback control system in order to maintain a fixed horizontal speed while traversing various terrain slopes. Two DC motors will power the rear wheels of the vehicle, and their rotational speeds will be adjusted by designing and implementing a Proportional-Integral-Derivative (PID) controller. The system represents a single Degree of Freedom (DOF) control effort, with the control variable being the rover's horizontal speed. Essentially, regardless of the terrain, the rover will maintain its speed through the use of a PID controller that constantly controls the torque of the wheels.

The inspiration for this project is primarily tied to the rovers developed by the National Aeronautics and Space Administration (NASA) to navigate Mars terrain, which exhibits various levels of gradient due to the planet's inconsistent surface. Figure 1 reveals a render of NASA's Perseverance rover, which is actively operating on Mars. The importance of a seed control system for a Mars rover is immeasurable, as it is constantly going through unknown terrain. A speed control system can also save power, a resource that is scarce in harsh environments in deep, unexplored space. With an interest in outer space, a speed controller seemed like the best choice not only due to the impact it would have but also because of its versatility and applicability to so many other fields.

Figure 1. NASA's Mars Perseverance Rover

Overall, the project achieved its objective in modeling a rover model with a cruise control feedback system. The scope of the project evolved slightly from its initial conception as the team discovered new information during development and made adjustments to ensure accurate and meaningful results were obtained. This adaptable approach allowed the team to maintain technical feasibility within the project timeline while still meeting the overall project goals. In the end, the main goal was to achieve any sort of speed control, while the final result may not have been as effective or fine-tuned as was initially hoped for, the overarching objective of the experiment was still achieved.

II. Introduction

Cruise control systems are widely used in aerospace applications to maintain a desired velocity without continuous input from a given operator. In the context of a rover vehicle, cruise control represents the ability of the system to sustain a constant speed by automatically adjusting motor input, which in turn induces torque. Modern applications of cruise control commonly utilize closed-loop feedback control, typically made up of a PID controller. This type of controller minimizes error between the target value and measured speed. These systems are foundational in the development of numerous aerospace vehicles and platforms. Figure 2 shows a block diagram for a typical simple control structure, which is composed of a controller block that operates on the error signal and a plant block that represents the dynamics of the system. In this project, the rover model constitutes the plant and its system dynamics will be modeled in the next section.

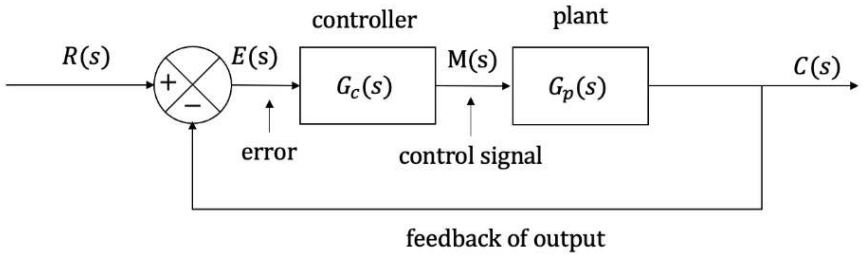

Figure 2. Typical Control Architecture Block Diagram

For planetary exploration, such as with Mars rovers, maintaining a constant speed helps manage torque levels, power consumption, and navigation over unpredictable terrain. This project builds on these principles by designing and implementing a cruise control system for a small-scale rover to simulate operation over sloped terrain, which ultimately reflects real-world, industry challenges. Actual planetary rovers exhibit more than one DOF and must process vast amounts of data from various sensors. For instance, NASA's Curiosity rover possesses 43 DOF, which are distributed across numerous components [1]. More information about the Curiosity rover and its hardware was also researched [2] to give a holistic approach to the experiment. Another research paper used to confirm the importance of this experiment was another about the Curiosity rover, and the already existing terrain adaptive features it possesses [3]. However, this project allows the team to gain insight into the procedure used to design and implement a controller in a practical setting.

The experimental setup process begins by modeling the system dynamics of the rover. This involves finding the equations of motion that govern how the system operates and re-writing them in state-space form. From here, a PID controller design is brainstormed and the controller specifications are established. Then, simulations are performed to predict the behavior of the rover model and tune the controller to satisfy the specifications. This PID controller should ensure the rover model sustains a constant horizontal speed across various terrain slopes. Finally, physical hardware is implemented to construct the rover model and employ the aforementioned PID controller. The numerical test results are then obtained and the controller performance is evaluated based on the response.

III. System Dynamics Modeling

A. System Overview

The system being controlled is a small-scale rover model, intended to maintain a constant translational velocity across varying slopes of terrain. The rover is driven by two rear wheels powered by DC motors, and its dynamics are characterized by the interaction between the electric motor, rotational inertia of the wheels, translational mass of the rover body, and frictional forces from an assumed pavement surface. To study the response of the rover to voltage inputs applied to the motor, a dynamic model of the open-loop system was developed. This model includes both electrical and mechanical subsystems and captures how input voltage influences linear velocity at the wheel-ground interface.

The team wanted to be as realistic as possible in modeling the behavior of the system in simulation so that the closed-loop controller could be well designed and implemented in an accurate manner. Motor voltage is the control input available to the embedded controller, and the output of interest is linear speed. Therefore, a model that relates motor input voltage to translational velocity is essential. This model should incorporate internal motor dynamics, rotational metrics, and expected frictional forces. The system mass and overall architecture are known and relatively straightforward, allowing for an analytical modeling approach. The simplicity of the structure justifies neglecting effects such as suspension dynamics, weight transfer, and aerodynamic losses.

B. Mathematical Model

The dynamic model includes the electrical characteristics of the DC motor, rotational dynamics of the motor and wheels, and translation of the entire rover mass. The derivation of the combined open-loop transfer function from motor input voltage $V(s)$ to output linear velocity $v(s)$ is provided in the following steps:

Step 1: Motor Angular Velocity Transfer Function

The motor and wheel dynamics are given by the following equation, which incorporates the total inertia J_{total} (including motors and wheels), total damping b_{total} (from motors and friction), motor inductance L_m, motor resistance R_m, and motor torque constant K_m:

$$G_\omega(s) = \frac{\omega(s)}{V(s)} = \frac{2 \cdot K_m}{(J_{\text{total}} s + b_{\text{total}})(L_m s + R_m) + K_m^2} \tag{1}$$

Rolling resistance is modeled as a constant force proportional to the system weight as seen in Equation 2. In this equation, μ represents the rolling resistance coefficient and m_{total} reflects the system mass. This can then be used to estimate the translational viscous damping coefficient via Equation 3 by assuming a nominal value for speed. This b_f can then be converted in terms of angular damping by using Equation 4. Finally, the total damping b_{total} in the rotational domain is simply the sum of this frictional rover damping and the intrinsic motor damping.

$$F_{\text{frict, nom}} = \mu \cdot m_{\text{total}} \cdot g \tag{2}$$

$$b_f = \frac{F_{\text{frict, nom}}}{v_{\text{nom}}} \tag{3}$$

$$b_{\text{rover}} = r_{\text{rear}}^2 \cdot b_f \tag{4}$$

Step 2: Convert Angular to Linear Velocity

The rear wheels convert angular velocity to linear velocity through the following:

$$G_v(s) = \frac{v(s)}{V(s)} = \frac{G_\omega(s)}{r_{\text{rear}}} \tag{5}$$

Step 3: Account for Translational Mass

To reflect the translational dynamics of the chassis, the total mass of the system must be realized:

$$G(s) = \frac{v(s)}{V(s)} = \frac{2 \cdot K_m}{r_{rear} \cdot m_{total} \cdot \left((J_{\text{total}} s + b_{\text{total}})(L_m s + R_m) + K_m^2\right)} \tag{6}$$

This transfer function accounts for the dynamics of the motor, mechanical torque generation through the wheels, linear acceleration of the body mass, and frictional forces due to the interaction with the ground.

C. Assumptions and Approximations

Several assumptions were made to simplify the mathematical modeling of the system. First, it is assumed that the rover moves along a uniform pavement surface with a constant coefficient of kinetic friction. Second, only translational dynamics are considered, yaw characteristics are neglected. Third, the system is treated as a rigid body without complex suspension dynamics. Four, motor parameters such as inductance, resistance, and torque constant are assumed based on typical values for a small DC motor.

D. System Identification

Tables I, II, and III show the data collected during system identification. Table I provides the parameters associated with the rover wheels. Table II includes data relating to a small-sized DC motor. Finally, Table III reveals the total parameter summary used to numerically interpret the open-loop transfer function. Most of the data was gathered in lab by using scales and measuring tools, however some values needed to be assumed. In addition, Figure 3 provides a geometric diagram of the motor dimensions.

Table I. Wheel Parameters

Parameter	**Symbol**	**Value**
Front wheel diameter	d_{front}	49×10^{-3} m
Front wheel radius	r_{front}	24.5×10^{-3} m
Front wheel mass	m_{front}	38.57×10^{-3} kg
Front wheel inertia	J_{front}	9.439×10^{-6} kg $\cdot$ m^2
Rear wheel diameter	d_{rear}	52×10^{-3} m
Rear wheel radius	r_{rear}	26×10^{-3} m
Rear wheel mass	m_{rear}	31.45×10^{-3} kg
Rear wheel inertia	J_{rear}	1.304×10^{-5} kg $\cdot$ m^2

Table II. Motor Parameters

Parameter	**Symbol**	**Value**
Motor torque constant	K_m	0.1 Nm/A
Motor rotor inertia	J_m	1.0×10^{-6} kg $\cdot$ m^2
Motor internal damping	b_m	1.0×10^{-4} N $\cdot$ m $\cdot$ s/rad
Motor inductance	L_m	0.01 H
Motor resistance	R_m	10 Ω

Table III. Total System Parameters

Parameter	Symbol	Value
Total system mass	m_{total}	0.45287 kg
Total motor inertia (2 motors)	J_{motors}	$2 \times J_m = 2.0 \times 10^{-6}$ kg · m^2
Total wheel inertia (4 wheels)	J_{wheels}	$2 \times J_{\text{front}} + 2 \times J_{\text{rear}} = 4.496 \times 10^{-5}$ kg · m^2
Total rotational inertia	J_{total}	$J_{\text{motors}} + J_{\text{wheels}} = 4.696 \times 10^{-5}$ kg · m^2
Frictional damping at nominal speed	b_f	7.5539 N · s/m
Converted rotational damping	b_{rover}	5.114×10^{-3} N · m · s/rad
Total damping	b_{total}	5.214×10^{-3} N · m · s/rad

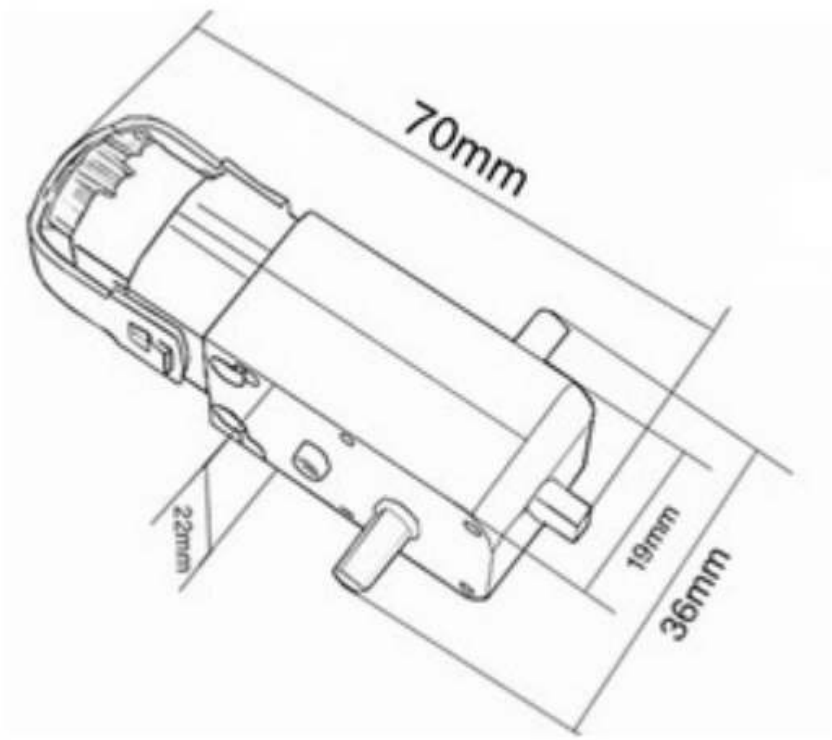

Figure 3. Dimensions of Motor [4]

The values provided in the tables above can then be plugged into Equation 6 to express the open-loop transfer function numerically. Equation 7 shows the numeric open-loop transfer function of the rover system. Figure 4 reveals the open-loop bode plot for the system, which indicates a stable and well-damped system.

$$G(s) = \frac{0.2}{5.529 \times 10^{-9} s^2 + 6.159 \times 10^{-6} s + 0.001101} \tag{7}$$

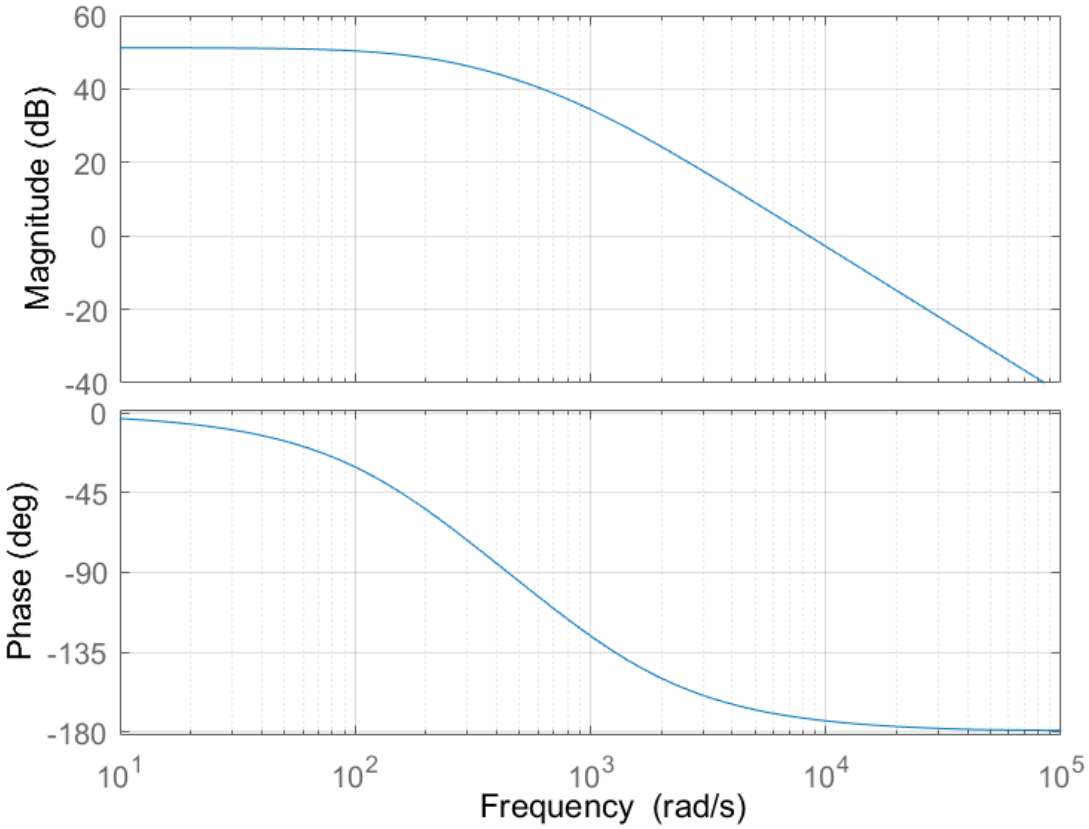

Figure 4. Open-Loop Bode Plot

IV. Controller Design

For the desired controller specifications, it was decided that the following requirements should be met:

Table IV. Controller Design Specifications

Parameter	Value
Rise Time	1s
Settling time	1.1s
Steady State Error	< 5%
Overshoot	< 1%

These choices were made with each specification's goal in mind. For the rise time, 1 second was chosen as reaching a desired speed quickly would be important in maintaining power consumption and steady speed without twitchy or sluggish response. A settling time of 1.1 seconds was chosen, as avoiding time spent not at speed could lead to unwanted power consumption or wear in parts. A steady state error of < 5% is more than enough to ensure that the rover does not destabilize due to changes in terrain or inclination. A percent overshoot of < 1% is acceptable as this rover's motions should be controlled and deliberate, ensuring that what is input as the desired speed is delivered as the desired speed.

A PID controller was utilized for this system. The following block diagram in Figure 5 represents the control system. However, as difficulties arose with implementing this controller design with the Arduino system on the rover, this control system was implemented entirely locally on the Arduino itself.

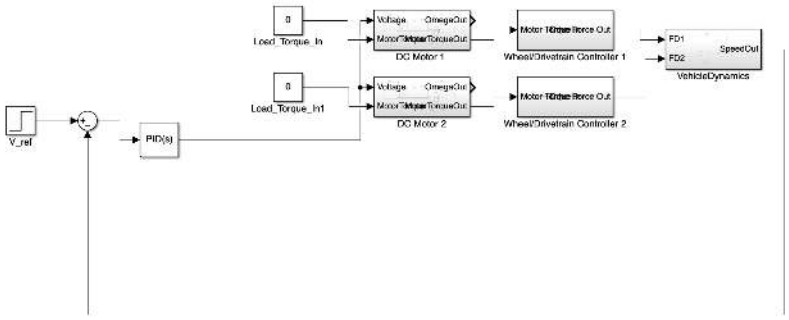

Figure 5. Block Diagram of cruise control PID controller

To design the controller, it is first important to identify the model that will serve as the plant for the controller. The plant was defined as follows:

$$G(s) = \frac{2 \cdot K_m}{r_{rear} \cdot m_{total} \cdot \left((J_{\text{total}} s + b_{\text{total}})(L_m s + R_m) + K_m^2\right)} \tag{8}$$

Now, interpreting the system as a second-order system, as seen in 9, it is possible to arrive at desired pole locations for the controller.

$$G_{cl} = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \tag{9}$$

Using percent overshoot and rise time, rough estimations of desired pole locations can be found.

$$M_p = e^{-\frac{\zeta\pi}{\sqrt{1-\zeta^2}}} \tag{10}$$

$$t_r \approx \frac{1.8}{\omega_n} \approx 1 \tag{11}$$

$$s_{1,2} = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2} \tag{12}$$

Using the standard form of a PID controller, as seen in Equation 13, a root locus can help move the location of new poles to their desired positions.

$$C(s) = K_p + \frac{K_i}{s} + K_d s \tag{13}$$

Once rough K_p, K_i, and K_d values have been found, further refining of these values through an iterative method is shown in Bode plot form in Figure 6. These iterations were made due to system limitations in our GPS monitoring of speed values, these system limitations being inconsistency in readouts from the GPS sensor, which causes large spikes or drops in data values, creating the following K_p, K_i, and K_d values:

Table V. Controller Design Results

Parameter	**Value**
K_p	0.9
K_i	1.9
K_d	0.07

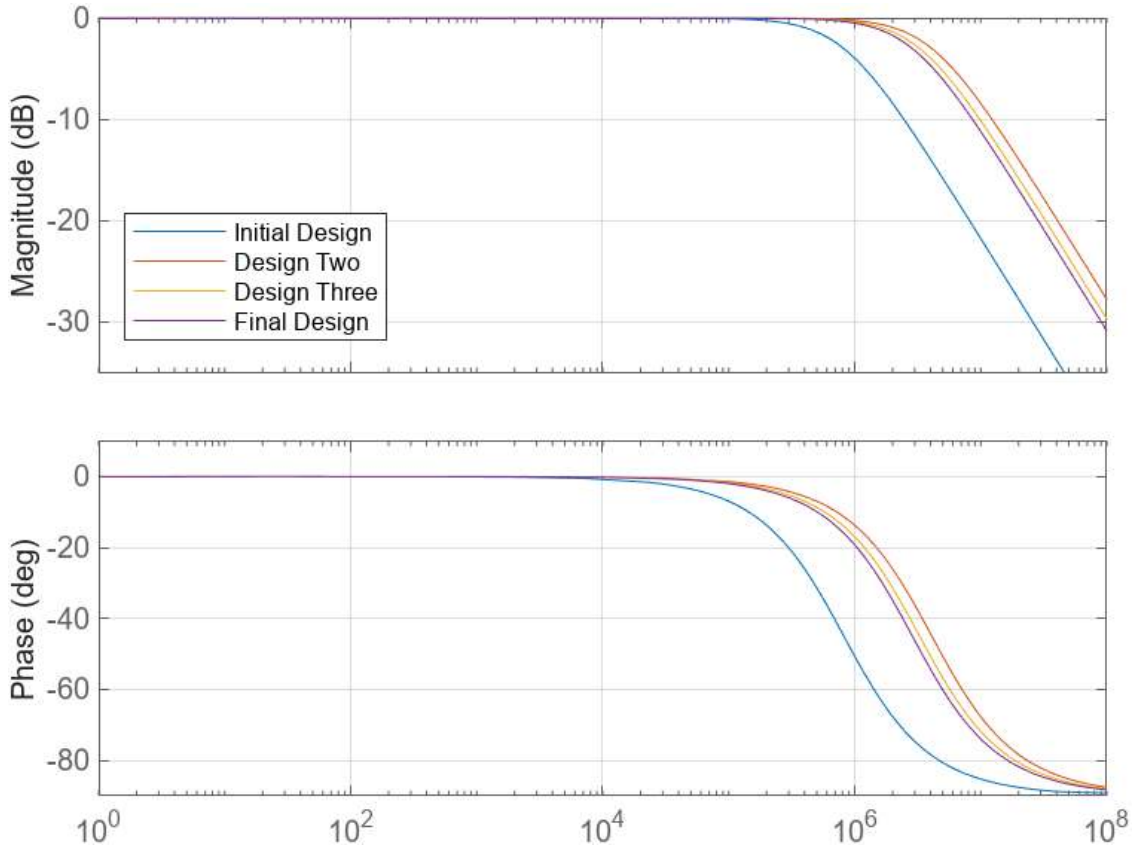

Figure 6. Bode plots of various iterations to K_p, K_i, and K_d values until desired performance was achieved

This resulted in a controller that surpassed our system's desired control specifications. The numerical results are as follows:

Table VI. Controller Design Results

Parameter	**Value**
Rise Time	0.0801s
Settling time	0.0981s
Steady State Error	$\approx 0\%$
Overshoot	$\approx 0\%$

With the values for K_p, K_i, and K_d selected, the closed-loop Bode plot can be found and is shown in Figure 7

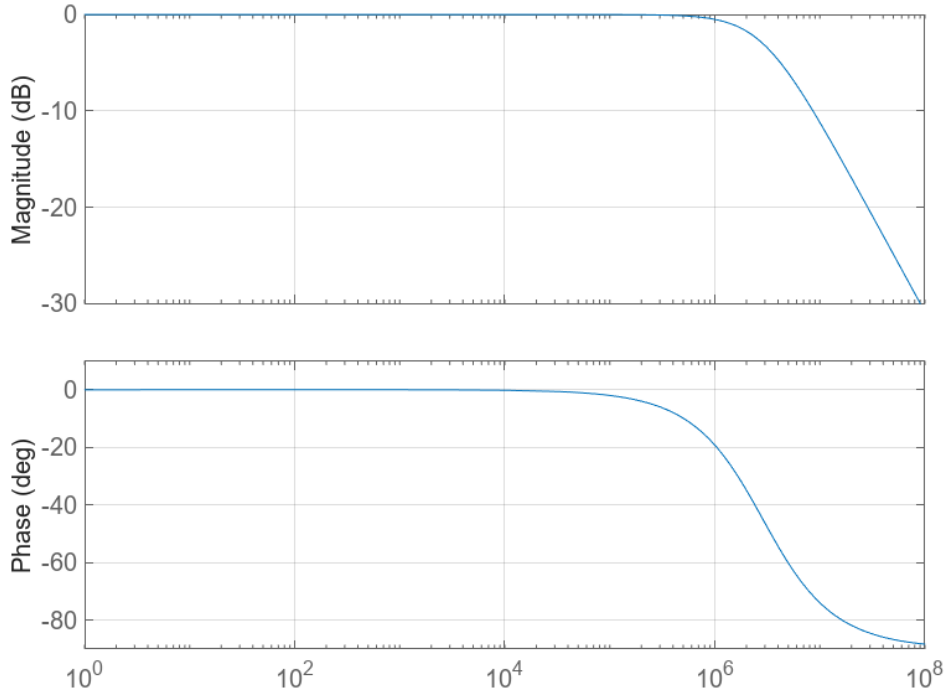

Figure 7. Closed-loop Bode plot

V. Simulation

The simulations for this project were performed with MATLAB and MATLAB Simulink. The purpose of these simulations was to create a model with a PID controller that was capable of meeting our response and settling time requirements in order for the system to be able to respond well enough to ensure that the cruise control would be smooth. Simulations were performed with the following Simulink model, shown in Figure 8

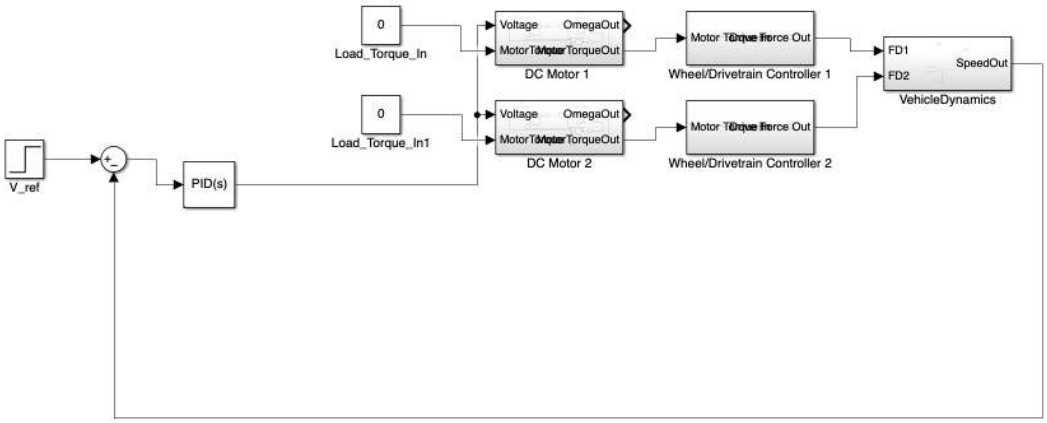

Figure 8. SIMULINK Block Diagram

We began with initial PID gains of $K_p = 0.4$, $K_i = 2.5$, and $K_d = 0.01$, then tuned them in three stages:

1) **Proportional tuning:** Set $K_i = 0$, $K_d = 0$, and increased K_p until small sustained oscillation appeared.
2) **Derivative tuning:** Raised K_d incrementally (up to 0.07) to damp the oscillations (see Figure 6).
3) **Integral tuning:** Introduced K_i (final value 1.9) to eliminate any residual steady-state error without reintroducing oscillation.

Table VII summarizes the performance at each iteration; the final gains $K_p = 0.9$, $K_i = 1.9$, and $K_d = 0.07$ yield a rise time of 0.08 s, settling time of 0.10 s, and overshoot below 1%, all within our specifications. Figure 9 shows the performance graphs for the various PID controller iterations.

Table VII. PID Controller Gains vs. Step Response Metrics

Iteration	K_p	K_i	K_d	Rise Time (s)	Settling Time (s)	Overshoot	Peak Time (s)
PID1	0.40	2.50	0.01	0.0081	0.0099	2.2×10^{-14}	4.44
PID2	0.40	0.50	0.05	0.0080	0.0098	0	5.00
PID3	0.40	0.25	0.10	0.0080	0.0098	0	5.00

Figure 11 shows the response of our simulated controller output.

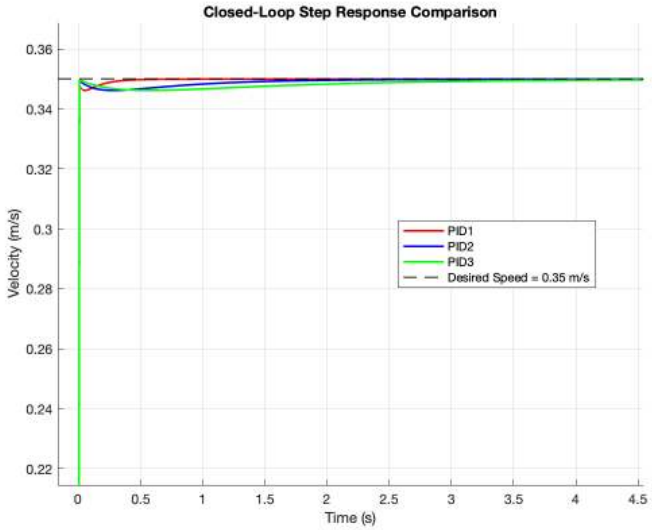

Figure 9. PID Controller Gain Iteration Progression

Figure 10. Step response comparing desired (0.35 m/s) vs. actual speed.

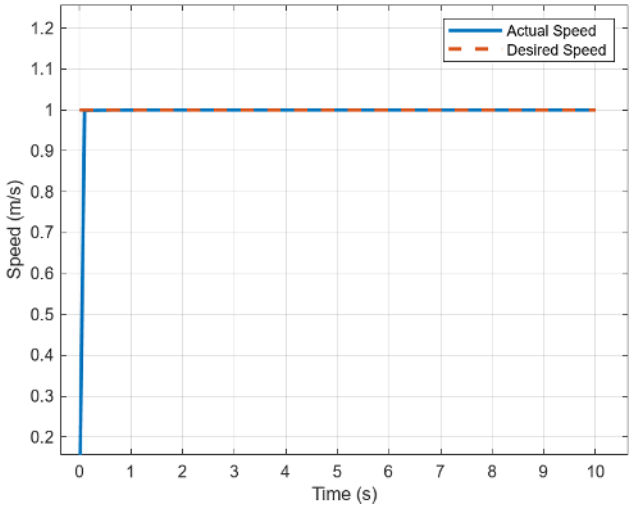

Figure 11. Speed response with cruise control PID

The above results demonstrate that our cruise control design meets rise-time, overshoot, and settling-time requirements under simulation.

VI. Hardware Implementation and Controller Evaluation

The physical hardware and electrical supplies utilized to construct the rover model is provided in Table VIII. Note that majority of these items were provided by the lab. The chassis kit was ordered online to make the fabrication process more efficient. In this way, the team could primarily focus on the simulation and controller design aspects of the project.

Table VIII. Physical Hardware and Electrical Components

Component	**Description**	**Quantity**
Arduino Uno	Microcontroller	1
GoouuuTech GPS Module	GPS sensor to measure speed	1
200 RPM Small DC Motor	Actuator to drive the rear wheels	2
L298N Motor Driver	To control DC motors	1
AA Battery	To power the motors and controller	4
Battery Case	To store batteries	4
Chassis Kit	To mount components and wheels	1
Wheels	Front and rear	4
Jumper Wires	To connect components	as needed

Even though the chassis kit was not manufactured by the team, much of the rover model still required assembly. The fabrication process of building the rover model is detailed in the steps provided below:

1. Ensure DC motors are attached to the two rear wheels.
2. Solder jumper wires to the DC motors for connection to the Arduino.
3. Attach all four wheels to the rover chassis, with the powered motors secured in the rear.
4. Secure Arduino and GPS to the top of chassis.
5. Connect components with jumper wires.

Figure 12 displays the diagram connecting the pins of the Arduino to specific parts of the rover assembly. Figure 13 reveals the complete rover model design after assembly. The rear, black wheels on the left side of the figure are the ones powered by the two DC motors. These motors are connected to pins 2-5, where pins 4 and 5 measure and regulate the speed of the rover. The motor controller is connected to an Arduino R3 Uno. The Arduino Uno is connected to a GoouuuTech GPS through the 12th and 13th Arduino pins. This GPS is most functional outdoors, which limited our group in certain tests.

The motor driver takes in power from 4 AA batteries, which are inserted into both the input voltage pin and the ground pin. This supplies power to the motor driver, which in turn powers the Arduino, the GPS module, and the DC motors. The aforementioned motors are connected to the side pins of the motor driver, as is shown in Figure 12.

The GoouuuTech GPS module is connected to the 5V pin on the Arduino. Meaning 5V of power is directly supplied to it. There is also a 3.3V pin on the Arduino board, which was used at first, however, the GPS connected faster and the signal was stronger with the use of the 5V pin as compared to the 3.3V pin. Figure 12 shows the configuration with the 5V pin connection, as this was the final design used in the working experiment. All sensors and motors were tested for different use cases. In the case of the motors, a variety of speeds were tested to ensure that the rover was able to move at a constant velocity from a constant DC motor torque. The GPS was also tested to make sure that quicker movements resulted in larger values for velocity. This was done as early in the experimental setup as possible to ensure the integrity of the hardware before more extensive work was done with it.

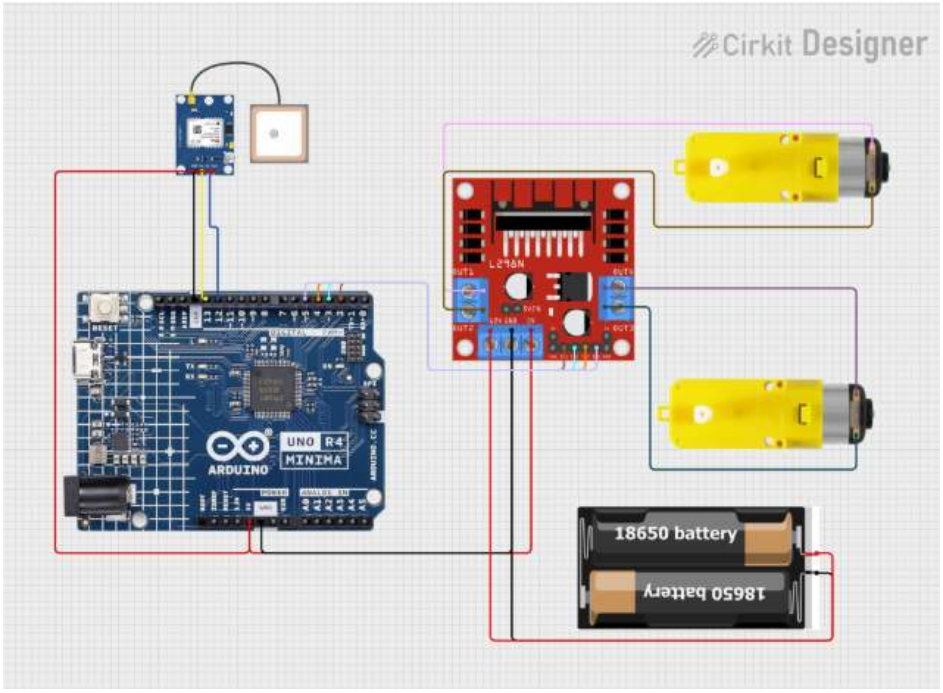

Figure 12. Circuit Diagram

Figure 13. Rover Design

The main goal of this experiment, as stated before, was to control the speed of the rover regardless of terrain. In essence, this means that regardless of the slope, the rover should travel at the same speed to not only ensure the rover's safety, but also to conserve power on declines.

The ideal design of the controller was described in the controller design section, and this was used as a starting point for the tuning of the controller used on the rover. Initially, the goal was to load this Simulink onto the Arduino and, hence, the rover; however, this proved extremely difficult and inefficient. For this reason, the controller was designed in the Arduino code instead, where variables were made for K_p, K_i, and K_d. A formula for a PID controller was then generated and used to create the speed control. Figure 14 shows the creation of the aforementioned gain values.

```
float Kp = 0.9;              // Proportional gain
float Ki = 0.04;              // Integral gain
float Kd = 0.07;             // Derivative gain
```

Figure 14. Gain variables created in the Arduino code

```
error = 13.3/39.3701 - currentSpeed;          // Proportional error
integral += error;                            // Integral term
float derivative = error - previousError;     // Derivative term
```

Figure 15. PID gain error values

Figure 15 shows the calculation of the PID value error terms. The first line shows the desired speed value, divided by the conversion factor from $\frac{in}{s}$ to $\frac{m}{s}$. The current speed is then subtracted from this value to get the overall error from the desired speed. Next, the integral variable is defined. The error is added to the current value for the integral variable and then set to equal itself once again. This makes sense when looking at Equation (14).

$$\int_0^t e(t)dt \tag{14}$$

As is common knowledge, an integral sums an infinite amount of infinitesimally small values, hence the reason why the integral term is written as the overall error added to itself consecutively.

The derivative term is the final line in Figure 15. This term is justified through the equation of derivative error, shown in Equation (15), and basic knowledge of what a derivative does.

$$\frac{d}{dt}e(t) \tag{15}$$

Arduino is not software capable of physically taking a derivative as Simulink and MATLAB are, so for this reason, the error term must be calculated with a slightly different method. Once again, it is common knowledge in Calculus that a derivative calculates the rate of change of a term. If the rate is constant and in a loop, the value is calculated each iteration, then it is known that the rate of change of time is simply one iteration every time. This is a constant and set rate of time, which is why the previous error can simply be subtracted from the current error.

Let's compare this to a real-world example that may be simpler to understand. Let's say that readings of the distance of a car are taken every 30 seconds. If the readings are taken every 30 seconds at a constant rate, then we can simply get the rate of change by dividing the distance change by one unit of measurement, which in this scenario would be 30 seconds. Again, this is exactly what is done in the last line of Figure 15, where the unit of time change is one loop iteration.

The serial output in the Arduino shows two different values, the first is the current velocity of a rover. This helps give an idea of how well the controller is doing its job. In the case of our rover, the GPS module prevented the effectiveness of these readings. The major drawback of the GPS is that it measures location, and thus speed, simply by latitude and longitude. This means that it does not take into consideration an increase in altitude in its calculations for speed. This is why tracking RPM would have been more effective had this been possible with the hardware. This is not to say the GPS

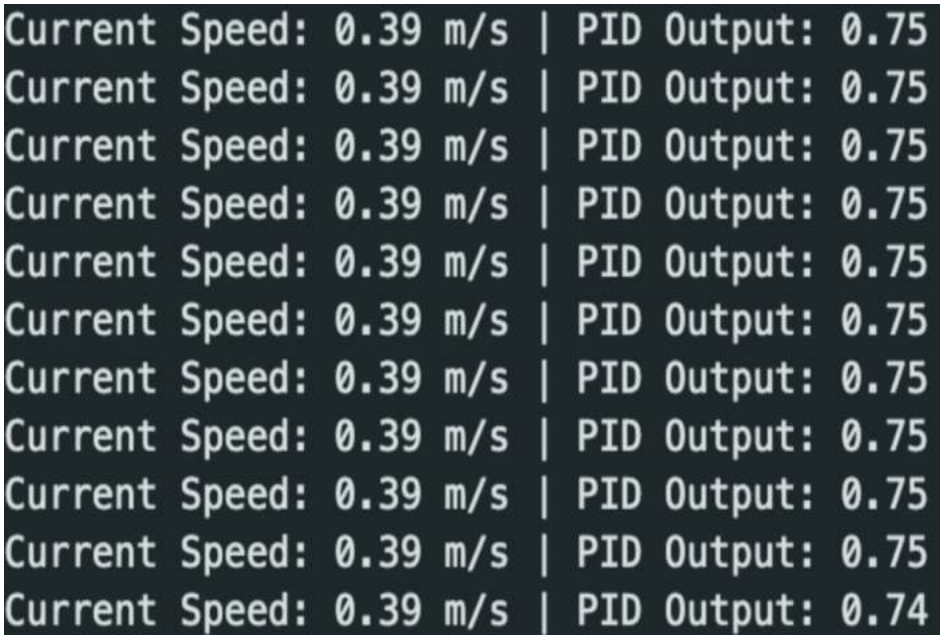

Figure 16. The serial output from the Arduino software

was unable to do its job; it is, by all means, capable of tracking speed accurately. However, where it fell short is in its inconsistency in getting the same reading multiple times. For this reason, points in the serial output where the values were consistent were primarily used in the analysis. In other words, some tolerance was accepted as a result of the equipment. Figure 16 shows the velocity values in $\frac{m}{s}$ alongside the PID values, which are the second value printed in the serial monitor.// The PID value in the serial monitor shows exactly what the controller is doing and how much control it is exerting upon the system. In Figure 16, the rover is going up an incline with a constant slope, so the value for the PID is constant. However, if the car is moving on a level surface and then transitions into an incline, it would be expected, and is proven through experiment and tuning, that the PID value would increase. Intuitively this makes sense as well, if the rover is travelling on level ground the controller would not have to do too much as the Arduino software would just be providing its constant torque for a level surface, however, as an incline is hit, in this example lets consider a positive incline, the car will slow down, and the controller will have to control the speed more than it was previously on the level surface. Hence why the PID value should be increasing alongside the incline. The code used for the calculation of the PID value printed to the serial monitor is shown in Figure 17.

```
// PID output
float pidOutput = Kp * error + Ki * integral + Kd * derivative;

// Set the motor speed based on the PID output
int pwmValue = constrain(115 + pidOutput, 0, 255);
```

Figure 17. PID value calculated for output in serial window of Arduino

The controller evaluation was fairly unsuccessful. It required countless hours of work and overcoming a multitude of obstacles to reach the results that we have previously outlined in our report. When we first received our parts from Table VIII, the package was missing part of the encoder system. Nothing that had the ability to read RPM based on the time of passage between the disk's windows was there. So, we switched to a magnet system to track RPM instead. The next issue was that it was impossible to properly set up the magnet system to accurately monitor the RPM on such a small rover. Yet again, we pivoted to using a GoouuuTech GPS sensor from another lab on campus. With this GPS, we had readings of velocity and a fully-functional rover. Due to all the moving pieces, the controller didn't fully work

because the GPS sensor doesn't record the speed nearly as quickly or precisely as required. The module wouldn't update nearly as quickly as an encoder would, ultimately meaning that velocity wasn't frequently measured.

Therefore, we were unable to see the fast, effective effects of the controller, and were also unable to provide quick live control to the torque commanded by Arduino. Although the GPS does a good job of tracking location, its velocity calculations were neither constant nor consistent. Sometimes, during testing, the results varied greatly. Furthermore, the only place the GPS module would work was on the top of a parking garage where there was minimal interference with the GPS module.

Lastly, Simulink doesn't perfectly align with Arduino. This means that creating a controller in Arduino initially would've been far more efficient. Our best way to improve this would be to use two different sensors in parallel, where one tracks RPM and the other measures speed to improve the accuracy of our results. This way, RPM can be used to provide quick and precise results to the controller, and the GPS can be used as a backup or as proof of the controller working.

Despite all the setbacks, a working controller and speed control system were achieved in the end. The system was not as consistent or effective as we would have liked, but with the obstacles we faced and the time we put in, it is something that we are proud of as a final product. One thing we wish we had been able to do would be to get plots of the controller actually working; this would have been a simple task had the Simulink been more interactive with the Arduino. However, as it stands, the serial plotter function in Arduino does not work. This means we could not get a plot of our PID values, which were printed to the serial window. The controller does work, however, further plots to prove this would have been more desirable had there not been so many moving pieces and pivots that had to be made to create a working controller at all.

In the end, we successfully created a driving rover that displays velocity values, as long as it is outdoors for satellite connection. Furthermore, ideal open and closed loop plots were generated for the case where the GPS module was more consistent and quicker with its velocity readings. A Simulink was also created for this ideal scenario, where the GPS was more effective in readings. Equations of motion and the weight and distance measurements for all rover components were determined and measured, and were used as the backbone for ideal controller generation and gain value selection.

The main difference between the ideal system and the experimental system was the value for integral gain. The value for integral gain in the ideal system was much larger. This was because, for a speed control system, the goal is to quickly settle back to the desired speed after a disturbance. In other words, the goal was to have a small steady-state error. As described before, the integral error is the accumulation of errors, and therefore, as time goes on, a stronger response is applied for a shorter amount of time when there is a large value for K_i. This is highlighted in Equation (16) which shows that a larger value for K_i actually significantly impacts the amount of control exerted on a system in a short amount of time, where the amount of time are the upper and lower bounds of the integral, and the control system is the function $u(t)$.

$$u(t) = K_i \int_0^t e(t) \tag{16}$$

Let's first consider the ideal system. Ideally, immediately after a disturbance in speed, the controller should bring the speed back to what is desired. This means having both a fast rise time and a settling time. Something that can be done with a large value for K_i, which ensures a quick controller response to a disturbance in the desired value for speed. However, as stated before, the problem with the GPS controller is that it jumps all over the place in its readings for the velocity. This means that a high value for K_i only serves to make the car jolt forward or backward quickly for each change in velocity. It is possible to implement a control for this as well, for example, something that looks at the previous value for velocity, and only prints the next if it has increased by more than a certain number, say 0.09 $\frac{m}{s}$. The problem with this, however, is that it completely defeats the purpose of the project; the car should control its speed on its own, and if small changes are tuned out, then the integrity of the project would no longer be intact.

The goal is to change the speed based on these small changes, not eliminate the reading of them entirely. In the end, this means that the integral gain value K_i for the experimental system had to be significantly lower than the ideal system. In fact, the experimental value for K_i was essentially zero. Without a low value for K_i in the experiment, the rover would jolt forward and back constantly, as the GPS was unable to make consistent readings of velocity. Naturally, this

meant the steady-state error would be higher, and the rover would react less quickly to a change in velocity. This meant that the rover was not as effective as we would have liked, although it does mean that the integrity of the controller and project was still maintained intact.

VII. Conclusion

The goal of this project was to build a small-scale Mars rover that utilizes a closed-loop feedback control system in order to maintain a fixed horizontal speed while traversing various terrain slopes. The controller type used was a PID controller. Through simulation and hardware testing, the system was capable of controlling the rover's speed accurately under ideal conditions. The controller was tuned to very specific performance specifications, where it yielded a rise time and settling time of less than 0.1 seconds. Also, it had about 0% steady-state error or overshoot.

As a group, we ran into a plethora of issues that we overcame transitioning from simulation to hardware implementation. The GPS sensor used to monitor velocity lacked the precision required for a real-time control system. Because of this, the controller's behavior was not consistent and we couldn't fully verify its effectiveness. Although the rover successfully responded to input and displayed velocity values, it couldn't regular speed reliably. Therefore, the hardware limitations impacted our overall system performance, even though the control system design was correct.

Valuable lessons were learned through this project. First, there was no seamless transition between Simulink and Arduino, which led to delays and implementation challenges. Second, acquiring the correct sensor is vital to control response and feedback quality. In the future, a wheel encoder working with the GPS, as well as a more suitable velocity sensor, would significantly improve controller performance. Despite the setbacks faced, our team constructed a functional rover, an accurate simulation model, and demonstrated an effective controller that meets design specifications under ideal conditions.

References

[1] Laboratory, N. J. P., "Curiosity Rover," , 2025. URL https://robotsguide.com/robots/curiosity, retrieved 8 April 2025.

[2] NASA, "Curiosity Science Instruments," , 2025. URL https://science.nasa.gov/mission/msl-curiosity/science-instruments/, accessed: 2025-04-22.

[3] Toupet, O., Biesiadecki, J., Rankin, A., Steffy, A., Meirion-Griffith, G., Levine, D., Schadegg, M., and Maimone, M., "Terrain-adaptive wheel speed control on the Curiosity Mars rover: Algorithm and flight results," , 2020. https://doi.org/10.1002/rob.21903.

[4] Seeed Technology Co., L., "TT Motor Dual Output Shaft Datasheet," , 2025. URL https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/694/114090050_Web.pdf?_gl=1*1v9ce6m*_up*MQ..*_gs*MQ..&gclid=CjwKCAjwtdi_BhACEiwA97y8BE5ZWUNQ_o2OhlwC7t-KG-ikdC7mBhPevRnIcg-MF2UHibb03jDJ_xoChukQAvD_BwE&gclsrc=aw.ds, retrieved 9 April 2025.